Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 7

по курсу «Языки и методы программирования»

«GUI сабскрайбер и ридер MQTT»

Студент группы ИУ9-21Б Воронков А. С.

Преподаватель Посевин Д. П.

Москва 2026

1 Цель

Целью лабораторной работы является приобретение навыка разработки на языке JAVA приложения выполняющего запись данных в топик публичного MQTT брокера и приложения чтения данных из топика.

2 Задание

1. Реализовать простой чатик

2. Реализовать приложение для подсчета евклидовой нормы вектора

3 Практическая реализация

Проект lab7: код файла Publisher.java

```
    import org.eclipse.paho.client.mqttv3.*;
import java.util.Scanner;


public class Publisher {
    private MqttClient client;
    private String topic = "IU/9/voronkov";

    public Publisher() throws MqttException {
        String broker = "tcp://broker.emqx.io:1883";
        //String broker = "tcp://192.168.43.204:1883";
        //String broker = "tcp://localhost:1883";
        String clientId = "voronkov_publisher";
        client = new MqttClient(broker, clientId);
        client.connect();
    }
        public void sendMessage(String text) {
            try{
                client.publish(topic, new MqttMessage(text.getBytes()));
            } catch (MqttException e) {
                e.printStackTrace();
            }
        }
    }
```

Проект lab7: код файла Subscriber.java

```
    import org.eclipse.paho.client.mqttv3.*;
import javax.swing.*;


public class Subscriber {
    private MqttClient client;
    private String topic = "IU/9/voronkov";
    private PictureForm1 gui;

    public Subscriber(PictureForm1 gui  ) throws MqttException{
        String broker = "tcp://broker.emqx.io:1883";
        //String broker = "tcp://192.168.43.204:1883";
        //String broker = "tcp://localhost:1883";
        String clientId = "voronkov_subscriber";
        try {
            this.client = new MqttClient(broker, clientId);
            MqttConnectOptions opts = new MqttConnectOptions();
```

```java

            client.setCallback(new MqttCallback() {
                public void messageArrived(String topic, MqttMessage message) throws Exception {
                    String mes = new String(message.getPayload());
                    System.out.println("message content: " + mes);
                    try {
                        SwingUtilities.invokeLater(() -> {
                            gui.updateLog(mes);
                        });

                    } catch (NumberFormatException e) {
                        System.out.println("Incorrect input");
                    }
                }

                public void connectionLost(Throwable cause) {
                    System.out.println("connectionLost: " + cause.getMessage());
                }

                public void deliveryComplete(IMqttDeliveryToken token) {
                    System.out.println("deliveryComplete: " + token.isComplete());
                }
            });

            System.out.println("Connecting to broker...");
            client.connect(opts);

            client.subscribe(topic);
            System.out.println("waiting");
        } catch (MqttException e) {
            System.err.println("            MQTT: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

Проект lab7: код файла Subscriber.java

```java
    import org.eclipse.paho.client.mqttv3.MqttException;

import javax.swing.*;
import java.awt.event.*;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.LocalTime;
```

```java

public class PictureForm1 {
    private JPanel mainPanel;
    private JSpinner xSpinner;
    private JSpinner ySpinner;
    private JSpinner zSpinner;
    private JTextArea textArea;
    private JTextPane textPane1;
    private JButton sendButton;
    private JTextField textField1;
    private JTextField textField2;
    private JButton deleteChatButton;
    private JTextArea textArea1;
    private Publisher publisher;
    private String username = "        ";
    public void updateLog(String text) {
        if (text.equals("delete")) {
            textArea.setText("");
        } else {
            textArea.append(gtTime() + " " + text + "\n");
            textArea.setCaretPosition(textArea.getDocument().getLength());
        }
    }

    public String gtTime() {
        LocalDate date = LocalDate.now();
        String formatted = date.format(DateTimeFormatter.ofPattern("dd.MM.yyyy"));

        LocalTime time = LocalTime.now();
        String formattedTime = time.format(DateTimeFormatter.ofPattern("HH:mm"));
        return formatted + " " + formattedTime;
    }


    public PictureForm1(Publisher publisher) {
        this.publisher = publisher;


        textField1.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                username = textField2.getText();
                String text = textField1.getText();
                if (!text.isEmpty()) {
```

```
                    publisher.sendMessage(username + ": " + text);
                    textField1.setText("");
                }
            }
        });
        sendButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                username = textField2.getText();
                publisher.sendMessage(textField1.getText());
                textField1.setText("");
            }
        });
        textField2.setText(username);
        deleteChatButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                publisher.sendMessage("delete");
            }
        });
    }


    public static void main(String[] args) {
        try {
            Publisher pub = new Publisher();
            PictureForm1 form = new PictureForm1(pub);
            JFrame frame = new JFrame("        ");
            frame.setContentPane(form.mainPanel);
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.pack();
            frame.setLocationRelativeTo(null);
            frame.setVisible(true);
            form.textField1.requestFocusInWindow();
            new Subscriber(form);
        } catch (MqttException e) {
            e.printStackTrace();
        }
    }
}
```

Проект rtfm7: код файла Publisher.java

```
import org.eclipse.paho.client.mqttv3.*;
import java.util.Scanner;


public class Publisher {
```

```java
    private MqttClient client;
    private String topic = "IU/9/voronkov";

    public Publisher() throws MqttException {
        String broker = "tcp://broker.emqx.io:1883";
        String clientId = "voronkov_publisher";
        client = new MqttClient(broker, clientId);
        client.connect();
    }
        public void sendMessage(String text) {
            try{
                client.publish(topic, new MqttMessage(text.getBytes()));
            } catch (MqttException e) {
                e.printStackTrace();
            }
        }
    }
```

Проект rtfm7: код файла Subscriber.java

```java
import org.eclipse.paho.client.mqttv3.*;
import javax.swing.*;


public class Subscriber {
    private MqttClient client;
    private String topic = "IU/9/voronkov";
    private PictureForm1 gui;

    public Subscriber(PictureForm1 gui  ) throws MqttException{
        String broker = "tcp://broker.emqx.io:1883";
        String clientId = "voronkov_subscriber";
        try {
            this.client = new MqttClient(broker, clientId);
            MqttConnectOptions opts = new MqttConnectOptions();

            client.setCallback(new MqttCallback() {
                public void messageArrived(String topic, MqttMessage message)
                    throws Exception {
                    String mes = new String(message.getPayload());
                    System.out.println("message content: " + mes);
                    String[] nums = mes.split(" ");
                    if (nums.length != 3) {
                        System.out.println("Incorrect input");
                        return;
                    }
                    try {
                        Vector vc = new Vector(mes);
```

```
                        double n = vc.norm();
                        System.out.println("Euclidian norm: " + n);
                        String result = "                    : " + mes + " |
                                                    : " + n;
                        SwingUtilities.invokeLater(() -> {
                            gui.updateLog(result);
                        });

                    } catch (NumberFormatException e) {
                        System.out.println("Incorrect input");
                    }
                }

                public void connectionLost(Throwable cause) {
                    System.out.println("connectionLost: " + cause.getMessage
                        ());
                }

                public void deliveryComplete(IMqttDeliveryToken token) {
                    System.out.println("deliveryComplete: " + token.
                        isComplete());
                }
            });

            System.out.println("Connecting to broker...");
            client.connect(opts);

            client.subscribe(topic);
            System.out.println("waiting");
        } catch (MqttException e) {
            System.err.println("                MQTT: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

Проект rtfm7: код файла Vector.java

```
    import java.lang.Math;

public class Vector {
    double x, y, z;
    public Vector (double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

```

```java
    public Vector (String s) {
        String[] nums = s.split(" ");
        double x, y, z;
        this.x = Double.parseDouble(nums[0]);
        this.y = Double.parseDouble(nums[1]);
        this.z = Double.parseDouble(nums[2]);
    }

    public void changeX(double x) { this.x = x;}
    public void changeY(double y) { this.y = y;}
    public void changeZ(double z) { this.z = z;}


    public double norm() {
        return Math.sqrt(x*x + y*y + z*z);
    }

    public String toString() {
        return x + " " + y + " " + z;
    }
}
```

Проект rtfm7: код файла PictureForm1.java

```java
import org.eclipse.paho.client.mqttv3.MqttException;

import javax.swing.*;
import javax.swing.event.ChangeEvent;
import javax.swing.event.ChangeListener;

public class PictureForm1 {
    private JPanel mainPanel;
    private JSpinner xSpinner;
    private JSpinner ySpinner;
    private JSpinner zSpinner;
    private JTextArea textArea;
    private Publisher publisher;
    public void updateLog(String text) {
        textArea.append(text + "\n");
        textArea.setCaretPosition(textArea.getDocument().getLength());
    }

    public PictureForm1(Publisher publisher) {
        this.publisher = publisher;
        xSpinner.setValue(10);
        ySpinner.setValue(10);
        zSpinner.setValue(10);
        Vector vector = new Vector(10, 10, 10);
```

```java
        xSpinner.addChangeListener(new ChangeListener() {
            @Override
            public void stateChanged(ChangeEvent e) {
                int x = (int)xSpinner.getValue();
                vector.changeX(x);
                publisher.sendMessage(vector.toString());
            }
        });
        ySpinner.addChangeListener(new ChangeListener() {
            @Override
            public void stateChanged(ChangeEvent e) {
                int y = (int)ySpinner.getValue();
                vector.changeY(y);
                publisher.sendMessage(vector.toString());
            }
        });
        zSpinner.addChangeListener(new ChangeListener() {
            @Override
            public void stateChanged(ChangeEvent e) {
                int z = (int)zSpinner.getValue();
                vector.changeZ(z);
                publisher.sendMessage(vector.toString());
            }
        });

    }


    public static void main(String[] args) {
        try {
            Publisher pub = new Publisher();
            PictureForm1 form = new PictureForm1(pub);
            JFrame frame = new JFrame("              ");
            frame.setContentPane(form.mainPanel);
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.pack();
            frame.setLocationRelativeTo(null);
            frame.setVisible(true);
            new Subscriber(form);
        } catch (MqttException e) {
            e.printStackTrace();
        }
    }
}
```

4 Результаты

В результате лабораторной работы было создано два приложения. Одно представляет из себя чат для общения нескольких пользователей через один топик. Второе приложение реализует отправку координат вектора и их чтение из соответствующего топика и подсчет евклидовой нормы вектора

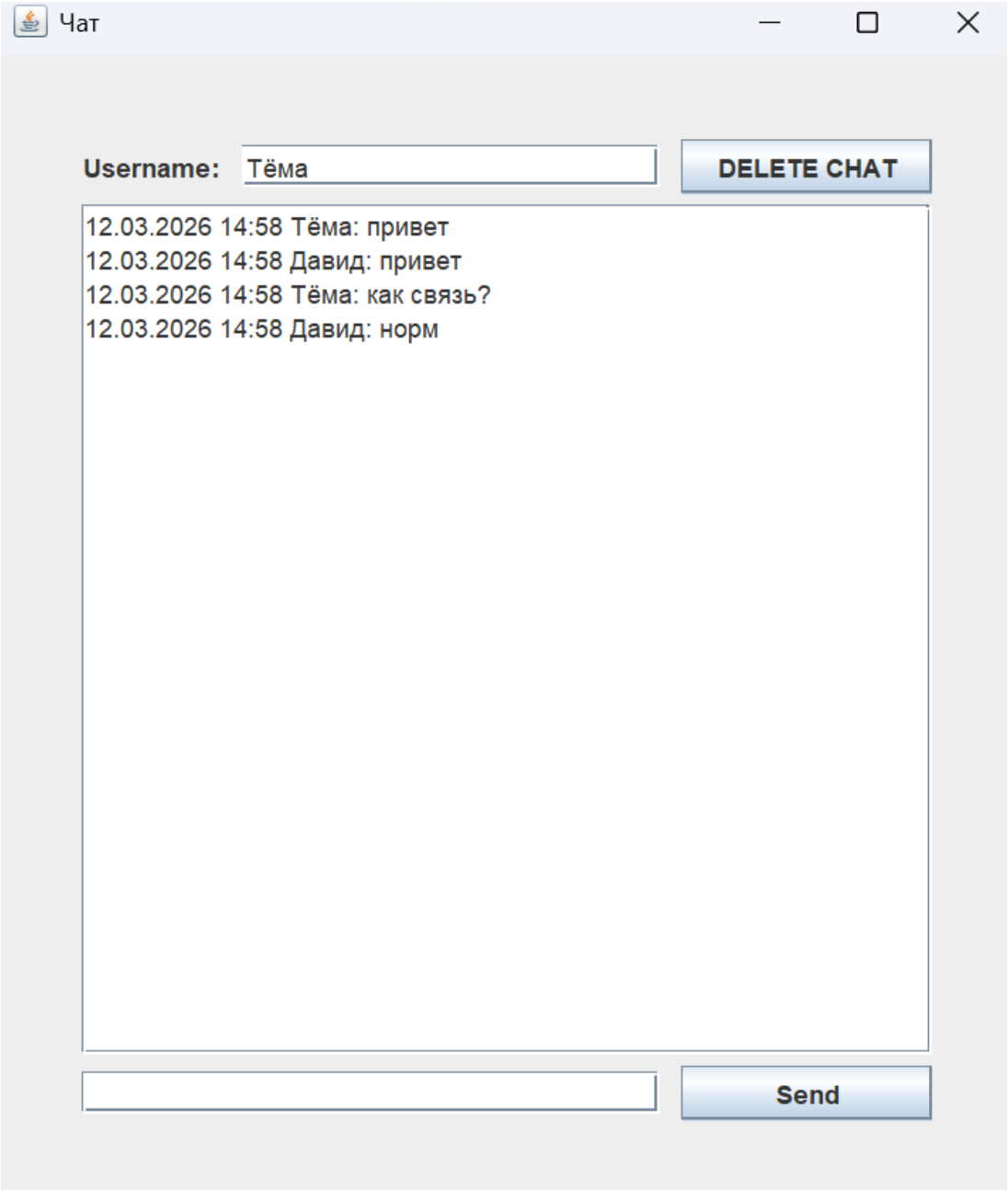

Рис. 1 — Визуализация работы чатика

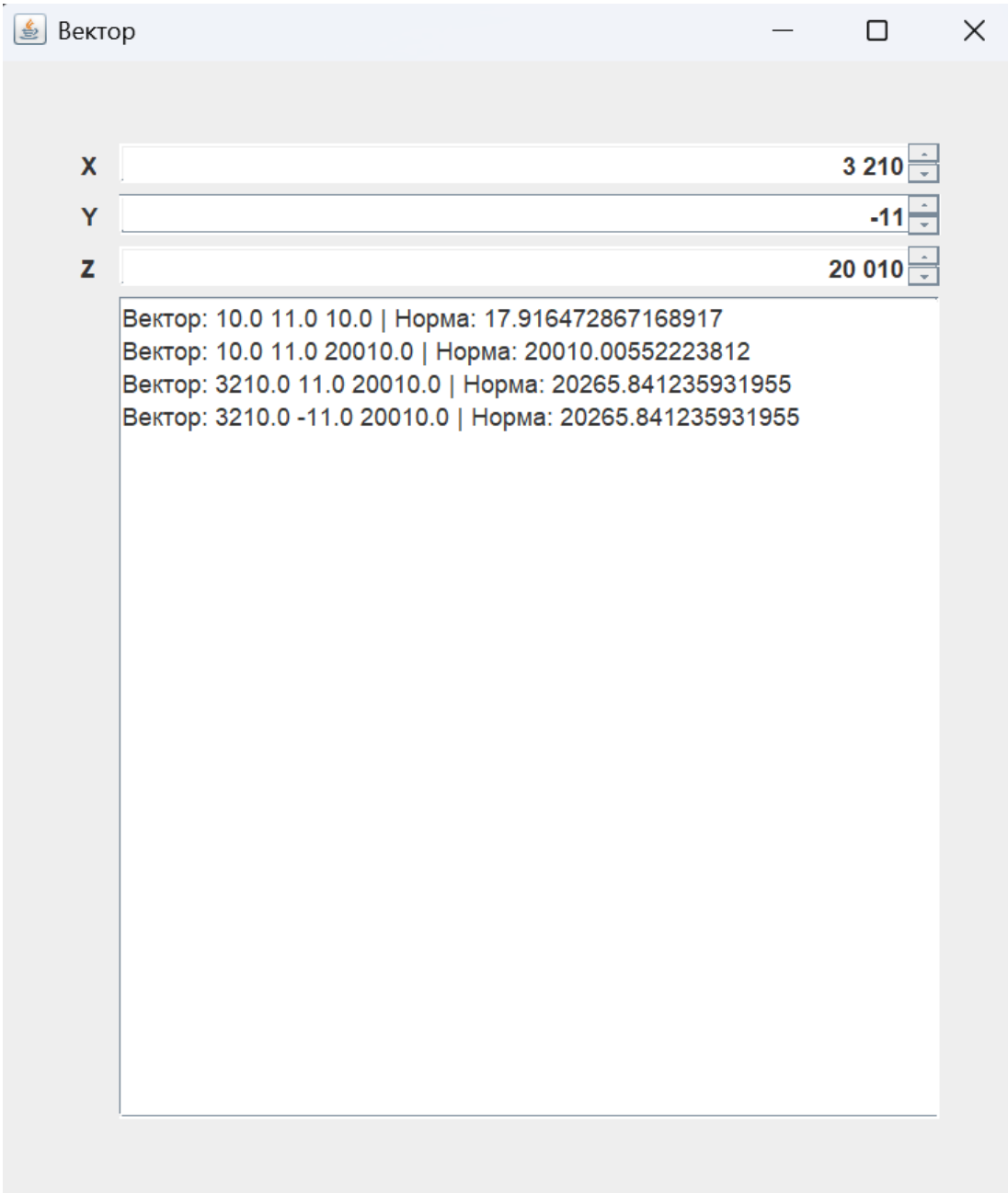

Рис. 2 — Визуализация работы приложения учета векторов

5 Вывод

В данной лабораторной работе были получены навыки работы с протоколом MQTT. Была написана своя читалка и писалка